

The Six-File Lab Record

A documentation structure for research projects that run long enough to forget themselves. Six files, one read policy each, and a test that fails when the prose stops matching reality.

THE ONE IDEA

Documents decay. Tests don't.

Every rule below exists because a document said something that had stopped being true, and nothing caught it. The six files are ordinary; the verifier is what makes them

hold. If you adopt one thing from this page, adopt the verifier.

STRUCTURE

Six files, six read policies

The split isn't by topic â€” it's by **when the file gets read**. That's what keeps the always-loaded file small and the permanent record complete.

CLAUDE.md

auto-loaded, every session

The contract. Mission, hard constraints, behavioral rules, wrap-up protocol. **Zero findings** â€” no results, no numbers, no history.

plan.md

read at session start

The dashboard. Current state in one paragraph, a ledger of completed work with results, what's in progress, what's queued, what's blocked.

terms.md

searched before naming anything

The glossary. Every metric with its definition, how it's computed, and **what silently changes its meaning**.

data.md

read before touching data or paths

The map. Datasets, schemas, splits, and a registry of every artifact â€” with a status column so dead things can't be quoted by accident.

memory.md

never read on startup Â· grep it

The lab notebook. Append-only, dated, permanent. Corrections are made **in place**, never by deletion.

bugs.md

grep by symptom

Symptom â†’ cause â†’ proven fix, indexed by subject. Separate from memory.md because bugs are looked up by symptom, not by date.

TEMPLATES

Copy these

Placeholders in <ANGLE BRACKETS> . Keep the structure; replace the domain.

CLAUDE.mdkeep under ~150 lines â€” it costs tokens every session

```
# <PROJECT> â€” Agent Contract

## Project Core
* **Mission:** <WHAT THIS PROJECT IS TRYING TO ESTABLISH, AND HOW SUCCESS IS MEASURED>
* **Environment:** <MACHINES, WHERE DATA LIVES, WHAT IS SHARED>
* **Stack:** <ENV ACTIVATION, KEY VERSIONS>. Not everything is <YOUR MAIN LANGUAGE> â€”
  check before assuming.
```

```

## Documentation Architecture
* `CLAUDE.md` â€” **auto-loaded every session.** This contract. **Zero findings.**
* `plan.md` â€” **read at session start.** Current state, active work, ledger.
* `terms.md` â€” **search before naming any metric.** Definitions + provenance.
* `data.md` â€” **read before touching data, runs or paths.**
* `memory.md` â€” **never read on startup; `grep` it.** Chronological ledger.
* `bugs.md` â€” **`grep` by symptom.** [Symptom] â†’ [Proven fix].

## Standing Constraints (hard rules; rationale lives in `memory.md`)
1. <THE RULE THAT PROTECTS AGAINST YOUR EASIEST WAY TO CHEAT A METRIC>
2. <WHICH DATA IS LIVE; WHAT IS OFF-LIMITS AND WHY>
3. <WHICH SUBSTRATE/MODEL/CONFIG IS CANONICAL â€” testing does not fan out>
4. Pre-registered endpoints do not change mid-phase.
5. A null is interpretable only if the test was POWERED **and** the intervention was VERIFIED TO HAVE LANDED â€” otherwise it is "uninformative", not "negative".
6. <WHAT IS HELD OUT, AND UNTIL WHEN>
7. <TWO MEASUREMENT PATHS THAT MUST NEVER BE MIXED IN ONE COMPARISON>
8. A validation holds only in the REGIME it was measured in. Re-derive any threshold or discrimination score when the inputs change.

## Agent Behavior & Prohibitions
* **Verify before acting.** Never guess paths, shapes, counts. Use `ls`/`grep`/`data.md`.
* **No sweeping changes.** No global `sed` or multi-file refactors without permission.
* **Strict documentation limits.** Do not create new doc files; these six are the set.
* **Prevent definition drift.** Search `terms.md` before naming a metric. Never invent synonyms.
* **Report the failure, not the workaround.** A missing tool must never silently become a negative result.

## IMPORTANT: Results Reporting Format
**Metrics are ROWS. Experiments are COLUMNS.** Never the transpose â€” a wide table of arms invites cherry-picking which metrics to show.
1. Report the full metric set every time, including rows that did not move. `n/a` with a reason; never an omitted row.
2. Label every number with its MEASUREMENT STAGE and n (see `terms.md`).
3. Row labels are the exact `terms.md` identifier. Mark gate metrics with `*`.
4. Order rows by importance: primary â†’ gates â†’ structure â†’ context â†’ demoted.
5. Carry a provenance line: <CHECKPOINT> Â· <INPUT SET> Â· n Â· <CONFIG>.
6. State the ceiling and the floor as their own columns.
7. Every table is followed by THREE things as BULLET LISTS: (a) one bullet per COLUMN, (b) one bullet per ROW, (c) a prose SYNTHESIS.
8. EVERY NUMBER IN PROSE MUST BE TRACEABLE TO A TABLE CELL, or name its source.

## Execution & Long-Running Tasks
* Anything over ~60s runs in a detached `tmux` session with a status sentinel:
  `tmux new-session -d -s <n> <cmd> > logs/<n>.log 2>&1; echo $? > logs/<n>.status`
* Poll the sentinel. Never report a run finished without reading it.
* **Any fan-out or workflow MUST assert its own completeness and fail loudly.**
  A script that filters results cannot tell "found nothing" from "ran nothing".
* Never `pkill -f <pattern>` where the pattern matches your own command line.

## IMPORTANT: The In-Place Correction Rule
`memory.md` is permanent. NEVER delete or overwrite an entry. When something is proven wrong:
1. Prepend `[INCORRECT] - ` to the original line, preserving the text.
2. Insert below: `[CORRECTION - YYYY-MM-DD]: ` with the new finding.

## YOU MUST EXECUTE: The Wrap-Up Protocol
1. **Archive to memory.** Hypothesis, method, provenance, results â†’ `memory.md`.
2. **Compress to the ledger.** One row in `plan.md`.
3. **Document fixes.** [Symptom] â†’ [Fix] to `bugs.md`.
4. **Define new terms.** Full `terms.md` entry.
5. **Register artifacts.** A `data.md` row for every new output.
6. **Reset the board.** Rewrite Current State and **bump `Last updated`**.
7. **Verify.** `python tests/test_docs_contract.py` must pass.

```

plan.md the only file that gets rewritten rather than appended

```
# plan.md " the dashboard

**Last updated:** YYYY-MM-DD

### Current State
<ONE PARAGRAPH. What is established, with numbers. What is closed and why. What the
binding constraint is right now. Written so someone who has never seen the project
knows what to do next.>

### Reporting contract
Two results are comparable only if their config stamps match on:
<AXIS 1> Â· <AXIS 2> Â· <AXIS 3>. Emit with `<YOUR SCORING SCRIPT>`; never hand-assemble.

### Ledger " <PHASE>
| ID | Intervention | Endpoint | n | Result | Verdict | memory |
|---|---|---|---|---|---|---|
| <ID> | <WHAT WAS DONE> | <terms.md NAME> | <N> | <VALUE> | <SIG / n.s. / UNINFORMATIVE> | YYYY-MM-DD |

**Provenance:** <CHECKPOINT> Â· <INPUT SET> Â· <CONFIG> Â· <WINDOW>

### In Progress
### [ID] <NAME>

Hypothesis: what would have to be true, stated so it can be wrong.
Reasoning: why this, why now, what it settles.
Technical: what actually runs.
Primary endpoint: ONE terms.md identifier + config.
n and power: n, from what estimate, powered to detect what.
MANIPULATION CHECK: the measurement proving the intervention LANDED.
Read BEFORE the endpoint.
Kill criterion: the pre-registered result that closes this line.

### Backlog
### [ID] <NAME> -€ NEXT
- **Why:** <WHAT IT SETTLES>
- **Prerequisite:** <WHAT MUST PASS FIRST>
- **Exit gate:** <WHAT RESULT MEANS IT WORKED>

### Blocked
| Item | Blocked on |
|---|---|

### Dropped " with reasons
| Item | Why |
|---|---|
```

terms.md the two fields that matter are Changes meaning with and Status

```
# terms.md " the glossary

Search this before defining a metric or labelling a table row.
Tags: [evaluation] [method] [gate] [diagnostic] [dataset] [training]

### <metric_identifier> [evaluation] [gate]
- **Is:** one sentence, unambiguous.
- **Computed by:** file:symbol " and the call that produces it.
- **CHANGES MEANING WITH:** <THE CONFIG THAT SILENTLY ALTERS THIS NUMBER UNDER THE
  SAME NAME.> This is the field that prevents the worst class of error.
- **Valid vs:** what it may be compared against. Everything else is invalid.
- **Status:** PRIMARY | SECONDARY | DIAGNOSTIC | DEMOTED | RETIRED
- **Aliases:** names seen in old docs or code keys. Do not use them.

### THE MEASUREMENT STAGES [method]
- **STAGE A " SELECTION.** Denominator: ALL outputs. "Is this a positive?"
  Every metric here must be defined for an output containing nothing.
```

- ****STAGE B** " CHARACTERISATION. **** Denominator: POSITIVES ONLY.** "Given a positive, what did we get?" These are misleading over all outputs, because a negative contributes a degenerate value that swamps the statistic.
- ****A Stage-B metric quoted over all outputs is not a weaker result** " it is a DIFFERENT and usually wrong quantity.

Retired
<NAME> " RETIRED YYYY-MM-DD. <WHY, AND WHAT REPLACED IT.> Do not re-add.

data.mdthe STATUS column is the point

```
# data.md " the map

**Rule.** Every number is traceable to a (<ARTIFACT>, <INPUT>, <CONFIG>, n) tuple.
If it is not in this file, it is not quotable.

Verified against disk YYYY-MM-DD. `tests/test_docs_contract.py` re-verifies.

## 1. Storage layout
| Root | Contents |
|---|---|

## 2. Record schema
| Field | Type | Notes |
|---|---|---|

š i, <ANY FIELD THAT IS NOT A UNIQUE KEY BUT LOOKS LIKE ONE. State the collision policy.>

## 3. Datasets " LIVE
| Name | n | Construction | Leakage control |
|---|---|---|---|

## 4. Datasets " DEPRECATED (DO NOT USE)
| Path | Why | Status |
|---|---|---|

## 5. Artifact registry
**Status key:** " LIVE (quotable) Â· š""| SUPERSEDED Â· š i, INVALID (never quote)

| Artifact | Date | Contents | Status |
|---|---|---|---|
```

memory.mdappend-only; newest at the bottom

```
# memory.md " the linear ledger

**DO NOT read on startup.** `grep` it.
grep -n "^## 20" memory.md # list entries
grep -n "INCORRECT|CORRECTION" memory.md # what we got wrong

## Rules
1. **Append only.** Never delete, never overwrite.
2. **In-place correction.** Prepend `[INCORRECT] - `; insert
  `[CORRECTION - YYYY-MM-DD]: ` below. The wrong version stays " it is what makes
  the reasoning legible later.
3. **Provenance or it didn't happen.**
4. Metric names come from `terms.md`.

---

## YYYY-MM-DD " <INTERVENTION | DECISION | FINDING>: <ONE-LINE VERDICT>
- **Hypothesis:** <STATED SO IT COULD BE WRONG>
- **Method:** <WHAT RAN>
- **Provenance:** <ARTIFACT> Â· <INPUT> Â· n Â· <CONFIG>
```

```
| metric | arm A | arm B | ceiling | floor |
|---|---|---|---|---|
```

```
**Per column:** <ONE BULLET EACH>
**Per row:** <ONE BULLET EACH>
**Synthesis:** <WHAT THEY SAY TOGETHER; WHAT THIS DOES NOT SHOW>
```

âš i, **What this does NOT establish:** <THE OVER-CLAIM SOMEONE WOULD MAKE FROM IT>

<!-- APPEND NEW ENTRIES BELOW THIS LINE -->

bugs.mdindexed by subject, not by date

```
# bugs.md â€” symptom â†’ proven fix
```

DO NOT read on startup. `grep` by symptom.

Different retrieval key from `memory.md`: results are looked up by DATE, bugs by SYMPTOM. Folding them together scatters one symptom across years.

âš i, The highest-severity class is **SILENT DEGRADATION** â€” a resource missing, a gate defaulting to its passing value, a flag accepted and ignored. These produce NUMBERS, not errors. When you fix one, record **what the wrong number looked like**, because that is what the next person will recognise.

```
### <SUBJECT â€” e.g. Data pipeline>
- **[YYYY-MM-DD] <ONE-LINE TITLE>**
  **[Symptom]** what you actually observe â€” the error text, or the wrong number.
  **[Cause]** the real mechanism, with file:line.
  **[Proven fix]** what actually worked. Not what should work.
  **[Severity]** note here if it corrupted a result.
```

```
### <SUBJECT â€” e.g. Agent / shell self-traps>
```

THE PART THAT MAKES IT WORK

The verifier

One test file, run at the end of every session. It checks the docs against the code and the disk. Without it, all of the above is a filing convention that decays in a month.

- Every metric the code computes has a `terms.md` entry â€” parse metric keys from your scoring module, diff against the glossary.
- Entries carry the required fields â€” Is / Computed by / Status. A retired metric needs only Status.
- The primary metric states what changes its meaning â€” the field that would have caught your worst error.
- No demoted or retired metric appears as a live endpoint in `plan.md`.
- Every path in `data.md` exists on disk and record counts match.
- Every artifact directory is registered â€” unregistered outputs are unattributable.
- No deprecated dataset is referenced from the active plan.
- Scored outputs stamp their config â€” a rate whose config is unstated is not a result.
- Corrections are well-formed â€” every `[INCORRECT]` has a dated `[CORRECTION]` below it.
- `plan.md` is not older than the newest `memory.md` entry â€” catches "results archived, board never reset".
- The governor carries no findings â€” no numbers, no results in the always-loaded file.

- Nothing was lost in migration** – if you're adopting this on an existing project, assert the old docs survive.

Make it runnable standalone (`python tests/test_docs_contract.py`), have checks **skip** rather than fail when a data mount is absent, and wire it into whatever runs your other tests. Use `WARN` for things you want visible but not blocking; `FAIL` for anything that would let a wrong number be quoted.

PROVENANCE

Rules, and the failures that produced them

None of these are general advice. Each one is a scar. They are the reason the framework looks the way it does, and they're the parts most worth keeping when you adapt it.

Metrics need a Changes meaning with field.

CAME FROM

A scoring function took a class argument and ignored it. The same metric name meant two different quantities. It inverted a headline result – and the bug was found, documented, and then left in the code for five days.

Corrections happen in place. Never delete.

CAME FROM

Four separate results were retracted or reversed. Each one is only legible now because the wrong version sits directly above the correction.

A null needs a manipulation check.

CAME FROM

A full training run returned an uninterpretable null because the treatment never landed – a flag was accepted, recorded as applied, and silently did nothing. Discovered only afterwards.

Validations expire when the regime changes.

CAME FROM

Three discrimination scores that justified an entire metric hierarchy collapsed to chance when re-tested under the configuration actually in use. One of them inverted.

Aggregate validation – per-component correctness.

CAME FROM

A detector validated at 25%– discrimination against its control while half of it measured the wrong thing entirely. Nobody had looked at which components were firing.

Measure sensitivity on real positives before adopting a metric.

CAME FROM

A metric was adopted, optimised against, and reported for days before anyone checked that it fires on only 8% of genuine positives.

Fan-outs must assert their own completeness.

CAME FROM

A parallel job returned a clean-looking zero. Every worker had crashed. "Found nothing" and "ran nothing" were indistinguishable in the return value.

Every number in prose traces to a table cell.

CAME FROM

A synthesis paragraph quoted figures from a different configuration than the table above it. The reader could not locate them and had no way to check the claim.

Name the denominator.

CAME FROM

A metric read 0.000 and was interpreted as "nothing resembles the reference". It actually meant "most outputs are negatives" – the statistic was reporting the hit rate, not similarity.

Zero-claims survive noise; ceilings don't.

CAME FROM

When a contaminated detector was thrown out, the finding "we produce none of these" survived – zero against a superset implies zero against the subset – while every reference rate it had produced was retracted.

ADOPTION

Standing it up on a new project

1

Write `terms.md` and `data.md` first. They carry the anti-drift load and they're the only two that can't be generated from existing material. Budget real time.

2

Seed `memory.md` by import, not rewrite. Concatenate whatever history exists under an `UNEDITED ARCHIVE` banner. Rewriting history is where errors get introduced.

3

Strip the governor to rules only. Move every finding out of the always-loaded file. In the project this came from, that file went from 362 lines to 130 – mostly by deleting results that belonged in the ledger.

4

Write the verifier before you need it. Start with three checks: metrics defined, paths exist, plan not stale. Add one every time something drifts.

5

Let it fail loudly the first time. A verifier that passes on day one probably isn't checking anything. Ours found a leaky dataset, a case collision, and 6,600 orphaned records on its first run.

Two things to resist. Don't add a seventh file – every candidate is a section in one of the six. And don't let the governor grow: when it hits its cap, **cut a section**, don't raise the limit.

Derived from a genome-modelling research project over ~40 sessions. Every rule on this page traces to a specific, dated failure in that project's `memory.md`.